UNIVERZITET CRNE GORE
ELEKTROTEHNIČKI FAKULTET

Pregled teme master rada

Dizajn i implementacija jednostavnog asemblerskog prevodioca
za prostu arhitekturu (podskup RISC-V RV32I)

Milica Delibašić

Podgorica, oktobar 2026.

Ovaj dokument je kratak pregled teme, pripremljen radi prvog uvida. Nije skraćena verzija rada: sadrži ideju, ono što je do sada implementirano i provjereno, predloženu strukturu rada i spisak onoga što ostaje.

---

1 Tema i problem

Tema je projektovanje i implementacija jednostavnog *dvoprolaznog asemblera*: programa koji prihvata izvorni kod napisan na asemblerskom jeziku i prevodi ga u mašinski kod. Za ciljnu arhitekturu izabran je osnovni cjelobrojni skup instrukcija RISC-V arhitekture (RV32I), otvorene arhitekture sa malim brojem pravilnih formata instrukcija, koja se sve više koristi i u industriji i u nastavi.

Polazno zapažanje je sljedeće. Industrijski asembleri, poput GNU asemblera, pouzdano prevode programe, ali su veliki i složeni alati čija unutrašnja struktura nije pogodna za upoznavanje sa principima prevođenja. Obrazovni simulatori su usmjereni na *izvršavanje* programa, a sam postupak *prevođenja* ostaje u pozadini:

- nije vidljivo kako asembler rješava *reference unaprijed*, tj. upotrebu labele koja je definisana tek kasnije u programu;
- kodiranje instrukcije u bitska polja, posebno kod grana i skokova, gdje su bitovi pomjeraja ispremještani, prikazuje se samo kao gotova heksadecimalna riječ;
- faze prevodioca iz teorije (leksička, sintaksna, semantička analiza i generisanje koda) nisu jasno razdvojene u kodu postojećih alata.

Problem koji rad obrađuje je, dakle, kako napraviti asembler koji je dovoljno mali da se može u potpunosti opisati i razumjeti, a dovoljno potpun da prevodi smislene programe (labele, podaci, grananja, petlje, pristup memoriji), uz jasne poruke o greškama i uz mašinski kod koji se može provjeriti poređenjem sa referentnim asemblerom.

2 Istraživačka pitanja

Glavno pitanje. Na koji način se može projektovati i implementirati jednostavan dvoprolazni asemblerski prevodilac za podskup RISC-V arhitekture (RV32I) tako da generiše ispravan mašinski kod i daje razumljive dijagnostičke poruke o greškama?

P1. Koji minimalni skup instrukcija, direktiva i pseudo-instrukcija RV32I arhitekture je dovoljan za pisanje smislenih programa (aritmetika, grananje, petlje, pristup memoriji)?

P2. Kako se klasične faze prevodioca preslikavaju na arhitekturu dvoprolaznog asemblera i koje su posebnosti obrade referenci unaprijed kod grana i skokova?

P3. U kojoj mjeri se mašinski kod razvijenog asemblera poklapa sa kodom referentnog asemblera (GNU `as`) za isti ulazni program?

3 Šta je do sada implementirano

Rješenje se razvija u jeziku Python 3.12, bez spoljnih biblioteka, kao alat komandne linije (`python -m asm program.asm -o out/`). Svaka faza prevodioca je zaseban modul. Implementirani su leksički analizator, tabela instrukcija i registara, kodiranje svih šest formata instrukcija sa provjerom opsega, tabela simbola, zapis izlaznih fajlova i komandna linija. Parser i dva prolaza su projektovani (algoritam i gramatika su definisani), ali još nisu napisani.

Tabela 1: Obim podržanog instrukcijskog skupa

Format	**Grupe instrukcija**	**Broj**
R	aritmetika, logika, pomjeranja i poređenja nad registrima	10
I	aritmetika i logika sa konstantom, pomjeranja za konstantu, učitavanje iz memorije, skok kroz registar	15
S	upis u memoriju	3
B	uslovna grananja	6
U	gornja konstanta (`lui`, `auipc`)	2
J	bezuslovni skok (`jal`)	1
Nativnih instrukcija ukupno (implementirano)		37
Pseudo-instrukcija (`nop mv not j ret li la`; planirano)		7
Direktiva asemblera (`.text .data .word .byte` …; planirano)		7

Tabela 2: Stanje po modulima

Modul	Stanje	Uloga
`lexer.py`	gotovo	tekst → tokeni, sa linijom i kolonom svakog tokena
`isa.py`	gotovo	tabela instrukcija (format, opcode, funct3, funct7) i registara
`encode.py`	gotovo	pakovanje polja u 32-bitnu riječ za formate R/I/S/B/U/J
`symbols.py`	gotovo	tabela simbola, dupla i nedefinisana labela
`output.py`	gotovo	binarni (little-endian) i heksadecimalni izlaz
`cli.py`	gotovo	komandna linija
`parser.py`	sljedeće	tokeni → labele, direktive, instrukcije
`assembler.py`	planirano	prvi prolaz (adrese, simboli) i drugi prolaz (kodiranje)

4 Ideja na konkretnom primjeru

Pošto je cilj rada da postupak prevođenja bude jasan, ideja se najkraće objašnjava primjerom. Riječi mašinskog koda u ovom odjeljku proizvodi već implementirani modul za kodiranje.

Dva prolaza i reference unaprijed

```
    addi t0, x0, 0
    addi t1, x0, 10
loop:
    addi t0, t0, 1
    blt  t0, t1, loop
```

Adresa	Instrukcija	Riječ
`0x00`	`addi t0, x0, 0`	`0x00000293`
`0x04`	`addi t1, x0, 10`	`0x00A00313`
`0x08`	`addi t0, t0, 1`	`0x00128293`
`0x0C`	`blt t0, t1, loop`	`0xFE62CEE3`

U *prvom prolazu* asembler samo računa adrese: brojač lokacija raste za 4 po instrukciji, a labela `loop` upisuje se u tabelu simbola sa adresom `0x08`. Ništa se još ne prevodi. U *drugom prolazu*, kada su sve adrese poznate, svaka instrukcija se kodira. Za granu `blt` upisuje se *razlika* adresa, `0x08` $-$ `0x0C` $= -4$, a ne sama adresa labele (PC-relativno adresiranje). Da je labela definisana tek poslije grane, postupak bi bio isti, jer je prvi prolaz već završen.

Rastavljanje instrukcije na bitska polja

Instrukcija `add x3, x1, x2` kodira se u riječ `0x002081B3`, po R formatu:

funct7	rs2	rs1	funct3	rd	opcode
`0000000`	`00010`	`00001`	`000`	`00011`	`0110011`
0	2 (`x2`)	1 (`x1`)	0	3 (`x3`)	`0x33`

Kod grane `blt t0, t1, loop` pomjeraj -4 zapisuje se kao 13-bitni broj, a njegovi bitovi se raspoređuju po B formatu na neuzastopna mjesta u riječi:

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
1	111111	00110	00101	100	1110	1	1100011

Dijagnostika sa tačnom lokacijom

Svaka greška nosi fajl, liniju i kolonu i prijavljuje se u fazi u kojoj nastaje. Primjeri stvarnog izlaza trenutne implementacije:

```
primjer.asm:1:14: lexical error: invalid integer literal '12abc'
primjer.asm:1:9: lexical error: unterminated string literal
primjer.asm:4:5: semantic error: immediate 5000 out of range (-2048..2047)
primjer.asm:9:5: semantic error: branch offset 4100 out of range (-4096..4095)
primjer.asm:12:5: semantic error: shift amount 40 out of range (0..31)
```

5 Verifikacija

Ispravnost se provjerava automatizovanim testovima (`pytest`). Očekivane vrijednosti kodiranja izvedene su nezavisno, iz definicije bitskih polja u specifikaciji RISC-V, a ne iz izlaza same implementacije, da bi test mogao otkriti grešku, a ne samo promjenu ponašanja. Za grane je svaki od 4096 mogućih pomjeraja kodiran i zatim vraćen nezavisnim dekoderom.

Tabela 3: Automatizovani testovi

Skup	Broj	Predmet provjere
Leksička analiza	50	tokeni, brojevi, stringovi, lokacije, greške
Tabela instrukcija	74	polja prema specifikaciji, jedinstvenost kodiranja, registri
Kodiranje instrukcija	85	poznate riječi svih šest formata, granice opsega
Paket i komandna linija	8	izlazni fajlovi, poruke, tabela simbola
Ukupno	217	svi prolaze

Kada parser i dva prolaza budu završeni, dodaju se integracioni testovi nad kompletnim programima (petlja, grananje, rad sa memorijom, poziv funkcije) i poređenje mašinskog koda sa izlazom GNU asemblera (`riscv64-unknown-elf-as`) za iste programe.

6 Predložena struktura rada i trenutni status

Struktura rada prati uputstvo za temu, a izgled je usklađen sa odobrenim master radovima Elektrotehničkog fakulteta (margine, font, prored, numeracija, redoslijed uvodnih strana). Pisani dio za sada postoji kao skelet: naslovi svih poglavlja i sekcija, sa bilješkama o sadržaju.

Tabela 4: Status po poglavljima

Poglavlje	Status	Napomena
Uvod	skelet	predmet, cilj i istraživačka pitanja preuzimaju se iz prijave
1. Osnove računarskih arhitektura i asemblera	skelet	literatura izabrana
2. Teorijske osnove prevodilaca	skelet	literatura izabrana
3. Specifikacija jednostavne arhitekture	djelimično	tabele formata i opkodova gotove
4. Projektovanje asemblerskog jezika	djelimično	gramatika (EBNF) i leksička pravila definisani
5. Dizajn i implementacija	prati kod	sekcije odgovaraju modulima
6. Praktična realizacija i primjeri	čeka kod	poslije drugog prolaza
7. Evaluacija i diskusija	čeka kod	poslije poređenja sa GNU `as`
Zaključak	—	na kraju
Prilozi	djelimično	izvorni kod i tabela opkodova uključuju se automatski

7 Šta ostaje

1. **Parser** — prevođenje tokena u labele, direktive i instrukcije po definisanoj gramatici.
2. **Prvi i drugi prolaz** — dodjela adresa, tabela simbola, kodiranje, proširenje pseudo-instrukcija (`li`, `la`, `j`, `ret`, …) i direktive za podatke.
3. **Izlaz za sekciju podataka** i završetak komandne linije (opciono: listing asembliranja).
4. **Integracioni testovi i poređenje sa GNU asemblerom** — odgovor na pitanje P3.
5. **Pisanje teksta rada** — teorijska poglavlja paralelno sa kodom, zatim evaluacija i zaključak.
6. Vaše korekcije na prijavi teme i na predloženoj strukturi rada.
7. Dopuna podataka: broj indeksa, tačan naziv studijskog programa i podaci o mentoru.

8 Literatura (izbor)

1. Aho A. V., Lam M. S., Sethi R., Ullman J. D., *Compilers: Principles, Techniques, and Tools*, 2. izd., Pearson/Addison-Wesley, 2006.
2. Beck L. L., *System Software: An Introduction to Systems Programming*, 3. izd., Addison-Wesley, 1997.
3. Waterman A., Asanović K. (ur.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, verzija 20191213, RISC-V Foundation, 2019.
4. Patterson D. A., Hennessy J. L., *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2. izd., Morgan Kaufmann, 2020.

5. Patterson D., Waterman A., *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon, 2017.
6. Waterman A., *Design of the RISC-V Instruction Set Architecture*, doktorska disertacija, University of California, Berkeley, 2016.
7. Salomon D., *Assemblers and Loaders*, Ellis Horwood, 1993.
8. Nystrom R., *Crafting Interpreters*, Genever Benning, 2021.
9. Petersen M. B., „Ripes: A Visual Computer Architecture Simulator", *IEEE/ACM Workshop on Computer Architecture Education (WCAE)*, 2021.
10. Free Software Foundation, *Using as: The GNU Assembler*, dokumentacija GNU Binutils.